

Experiment ‘p_5_5_repeat’ Results

June 4, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 1

Soft fails (resolution): 1

Verification attempts: 2

Problem Specification

Problem name: p_5_5_repeat

Natural language statement: Write a method that returns a string repeated n times. For example, repeat("ho", 3) returns "hohoho".

Method signature: p_5_5_repeat(str: string, n: int) returns (repeated_str: string)

Ensures

- $|repeated_str| == n * |str|$
- forall $i :: 0 \leq i < n \implies repeated_str[i * |str| .. (i + 1) * |str|] == str$

Requires

- $n \geq 0$

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that returns a string repeated n times. For example, `repeat("ho", 3)`
↪ returns "hohoho".

The signature should be:

`p_5_5_repeat(str: string, n: int)` returns `(repeated_str: string)`

The method should respect the following contract:

requires `n >= 0`, ensures `|repeated_str| == n * |str|`, ensures forall `i :: 0 <= i < n ==>`
↪ `repeated_str[i * |str| .. (i + 1) * |str|] == str`

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4OIZGqgCiFM2lkqZCUKKvdJ9CLR

Model: gpt-5.5-2026-04-23

Created at: 1780587351

Finish reason: stop

Usage: 378 tokens in, and 7061 tokens out

```
{
  repeated_str := "";
  var k := 0;
  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i :: 0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var prev := repeated_str;
    var kk := k;

    assert |prev| == kk * |str|;
    forall i | 0 <= i < kk
      ensures prev[i * |str| .. (i + 1) * |str|] == str
    {
      assert 0 <= i < k;
      assert prev == repeated_str;
      assert repeated_str[i * |str| .. (i + 1) * |str|] == str;
    }
  }
}
```

```

repeated_str := prev + str;

assert repeated_str == prev + str;
assert |repeated_str| == |prev| + |str|;
assert (kk + 1) * |str| == kk * |str| + |str|;
assert |repeated_str| == (kk + 1) * |str|;
assert |prev| <= |repeated_str|;
assert repeated_str[..|prev|] == prev;
assert repeated_str[|prev|..|repeated_str|] == str;

forall i | 0 <= i < kk + 1
  ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
{
  var a := i * |str|;
  var b := (i + 1) * |str|;
  assert 0 <= |str|;
  assert 0 <= a <= b;
  if i < kk {
    assert i + 1 <= kk;
    assert b <= kk * |str|;
    assert b <= |prev|;
    assert repeated_str[a..b] == repeated_str[..|prev|][a..b];
    assert repeated_str[..|prev|][a..b] == prev[a..b];
    assert prev[a..b] == str;
  } else {
    assert i == kk;
    assert a == |prev|;
    assert b == |repeated_str|;
    assert repeated_str[a..b] == str;
  }
}

k := k + 1;
assert k == kk + 1;
}
assert k == n;
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_5_5_repeat/program.dfy(1,7): Error: Verification of 'p_5_5_repeat' timed
  ↳ out after 31 seconds. (the limit can be increased using --verification-time-limit)
1 | method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
  | ~~~~~

```

Dafny program verifier finished with 1 verified, 0 errors, 1 time out

Correct the error in code by altering only the method body. Produce and show only the
→ Dafny body, including the curly braces that surround it. Do not show the signature
→ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4SFaRN2bW7VIhu59rVjJG070Tl8

Model: gpt-5.5-2026-04-23

Created at: 1780587567

Finish reason: stop

Usage: 1151 tokens in, and 5784 tokens out

```
{
  repeated_str := "";
  var k := 0;

  while k < n
    invariant 0 <= k <= n
    invariant |repeated_str| == k * |str|
    invariant forall i {:trigger repeated_str[i * |str| .. (i + 1) * |str|]} ::
      0 <= i < k ==> repeated_str[i * |str| .. (i + 1) * |str|] == str
  {
    var old := repeated_str;

    forall i | 0 <= i < k
      ensures old[i * |str| .. (i + 1) * |str|] == str
    {
      assert old == repeated_str;
      assert repeated_str[i * |str| .. (i + 1) * |str|] == str;
    }

    repeated_str := old + str;

    assert repeated_str == old + str;
    assert |old| == k * |str|;
    assert |repeated_str| == |old| + |str|;
    assert |repeated_str| == (k + 1) * |str|;
    assert repeated_str[..|old|] == old;
    assert repeated_str[|old|..] == str;

    forall i | 0 <= i < k + 1
      ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
    {
      var a := i * |str|;
      var b := (i + 1) * |str|;
      if i < k {
        assert i + 1 <= k;
        assert b <= k * |str|;
        assert b <= |old|;
      }
    }
  }
}
```

```

    assert repeated_str[a..b] == repeated_str[..|old|][a..b];
    assert repeated_str[..|old|][a..b] == old[a..b];
    assert old[a..b] == str;
  } else {
    assert i == k;
    assert a == |old|;
    assert b == |repeated_str|;
    assert repeated_str[a..b] == repeated_str[|old|..];
    assert repeated_str[|old|..] == str;
  }
}

k := k + 1;
}

assert k == n;
forall i | 0 <= i < n
  ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
{
  assert 0 <= i < k;
}
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

```

problem_data/p_5_5_repeat/program.dfy(15,8): Error: invalid Ident
|
15 | var old := repeated_str;
| ^

problem_data/p_5_5_repeat/program.dfy(18,17): Error: openparen expected
|
18 | ensures old[i * |str| .. (i + 1) * |str|] == str
| ^

problem_data/p_5_5_repeat/program.dfy(18,28): Error: rbracket expected
|
18 | ensures old[i * |str| .. (i + 1) * |str|] == str
| ^

3 parse errors detected in program.dfy

```

Correct the error in code by altering only the method body. Produce and show only the
 → Dafny body, including the curly braces that surround it. Do not show the signature
 → nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4UON14nLPqE0Fevw3tGlZ97N1kd

Model: gpt-5.5-2026-04-23

Created at: 1780587700

Finish reason: stop

Usage: 2039 tokens in, and 4588 tokens out

```
{
  if n == 0 {
    repeated_str := "";
  } else {
    var prev := p_5_5_repeat(str, n - 1);
    repeated_str := prev + str;

    assert |prev| == (n - 1) * |str|;
    assert |repeated_str| == |prev| + |str|;
    assert n * |str| == (n - 1) * |str| + |str|;
    assert |repeated_str| == n * |str|;

    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..] == str;

    forall i | 0 <= i < n
      ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
    {
      var a := i * |str|;
      var b := (i + 1) * |str|;
      assert 0 <= |str|;
      assert 0 <= a <= b;

      if i < n - 1 {
        assert 0 <= i < n - 1;
        assert i + 1 <= n - 1;
        assert b <= (n - 1) * |str|;
        assert b <= |prev|;
        assert repeated_str[a..b] == repeated_str[..|prev|][a..b];
        assert repeated_str[..|prev|][a..b] == prev[a..b];
        assert prev[a..b] == str;
      } else {
        assert i == n - 1;
        assert a == |prev|;
        assert b == |repeated_str|;
        assert repeated_str[a..b] == repeated_str[|prev|..];
        assert repeated_str[|prev|..] == str;
      }
    }
  }
}
```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```
problem_data/p_5_5_repeat/program.dfy(21,26): Error: upper bound below lower bound or
  ↳ above length of sequence
```

```
|
21 | ensures repeated_str[i * |str| .. (i + 1) * |str|] == str
| ~
```

```
problem_data/p_5_5_repeat/program.dfy(31,8): Error: assertion might not hold
```

```
|
31 | assert b <= (n - 1) * |str|;
| ~~~~~
```

Dafny program verifier finished with 1 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
↳ Dafny body, including the curly braces that surround it. Do not show the signature
↳ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn4WyURPTExJZm7yx6kES3G8JEVNY

Model: gpt-5.5-2026-04-23

Created at: 1780587860

Finish reason: stop

Usage: 2654 tokens in, and 8180 tokens out

```
{
  if n == 0 {
    repeated_str := "";
    assert |repeated_str| == 0;
    assert 0 * |str| == 0;
  } else {
    var L := |str|;
    var m := n - 1;
    assert 0 <= m;
    assert n == m + 1;

    var prev := p_5_5_repeat(str, m);
    repeated_str := prev + str;

    assert |prev| == m * L;
    assert |repeated_str| == |prev| + L;
    assert m * L + L == (m + 1) * L;
    assert |repeated_str| == (m + 1) * L;
    assert |repeated_str| == n * L;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..|repeated_str|] == str;
  }
}
```

```

forall i | 0 <= i < n
  ensures 0 <= i * L && i * L <= (i + 1) * L && (i + 1) * L <= |repeated_str| &&
    repeated_str[i * L .. (i + 1) * L] == str
{
  assert 0 <= L;
  assert 0 <= i;
  assert i + 1 <= n;
  assert 0 <= n - (i + 1);
  assert 0 <= i * L;
  assert (i + 1) * L == i * L + L;
  assert i * L <= (i + 1) * L;
  assert n * L == (i + 1) * L + (n - (i + 1)) * L;
  assert 0 <= (n - (i + 1)) * L;
  assert (i + 1) * L <= n * L;
  assert (i + 1) * L <= |repeated_str|;

  if i < m {
    assert 0 <= i < m;
    assert i + 1 <= m;
    assert 0 <= m - (i + 1);
    assert m * L == (i + 1) * L + (m - (i + 1)) * L;
    assert 0 <= (m - (i + 1)) * L;
    assert (i + 1) * L <= m * L;
    assert (i + 1) * L <= |prev|;
    assert prev[i * L .. (i + 1) * L] == str;
    assert repeated_str[i * L .. (i + 1) * L] == repeated_str[..|prev|][i * L .. (i +
      ↪ 1) * L];
    assert repeated_str[..|prev|][i * L .. (i + 1) * L] == prev[i * L .. (i + 1) * L];
    assert repeated_str[i * L .. (i + 1) * L] == str;
  } else {
    assert i == m;
    assert i * L == m * L;
    assert i * L == |prev|;
    assert i + 1 == m + 1;
    assert (i + 1) * L == (m + 1) * L;
    assert (i + 1) * L == |repeated_str|;
    assert repeated_str[i * L .. (i + 1) * L] == repeated_str[|prev|..|repeated_str|];
    assert repeated_str[i * L .. (i + 1) * L] == str;
  }
}

assert L == |str|;
assert |repeated_str| == n * |str|;
assert forall i :: 0 <= i < n ==>
  (0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |repeated_str|
    ↪ &&
    repeated_str[i * |str| .. (i + 1) * |str|] == str);
}
}

```


Final Program

```
method p_5_5_repeat(str: string, n: int) returns (repeated_str: string)
  requires n >= 0
  ensures |repeated_str| == n * |str|
  ensures forall i :: 0 <= i < n ==> repeated_str[i * |str| .. (i + 1) * |str| ] ==
    ↪ str
{
  if n == 0 {
    repeated_str := "";
    assert |repeated_str| == 0;
    assert 0 * |str| == 0;
  } else {
    var L := |str|;
    var m := n - 1;
    assert 0 <= m;
    assert n == m + 1;

    var prev := p_5_5_repeat(str, m);
    repeated_str := prev + str;

    assert |prev| == m * L;
    assert |repeated_str| == |prev| + L;
    assert m * L + L == (m + 1) * L;
    assert |repeated_str| == (m + 1) * L;
    assert |repeated_str| == n * L;
    assert repeated_str[..|prev|] == prev;
    assert repeated_str[|prev|..|repeated_str|] == str;

    forall i | 0 <= i < n
      ensures 0 <= i * L && i * L <= (i + 1) * L && (i + 1) * L <= |repeated_str| &&
        repeated_str[i * L .. (i + 1) * L] == str
    {
      assert 0 <= L;
      assert 0 <= i;
      assert i + 1 <= n;
      assert 0 <= n - (i + 1);
      assert 0 <= i * L;
      assert (i + 1) * L == i * L + L;
      assert i * L <= (i + 1) * L;
      assert n * L == (i + 1) * L + (n - (i + 1)) * L;
      assert 0 <= (n - (i + 1)) * L;
      assert (i + 1) * L <= n * L;
      assert (i + 1) * L <= |repeated_str|;

      if i < m {
        assert 0 <= i < m;
        assert i + 1 <= m;
        assert 0 <= m - (i + 1);
        assert m * L == (i + 1) * L + (m - (i + 1)) * L;
        assert 0 <= (m - (i + 1)) * L;
        assert (i + 1) * L <= m * L;
      }
    }
  }
}
```

```

    assert (i + 1) * L <= |prev|;
    assert prev[i * L .. (i + 1) * L] == str;
    assert repeated_str[i * L .. (i + 1) * L] == repeated_str[..|prev|][i * L .. (i +
    ↪ 1) * L];
    assert repeated_str[..|prev|][i * L .. (i + 1) * L] == prev[i * L .. (i + 1) * L];
    assert repeated_str[i * L .. (i + 1) * L] == str;
} else {
    assert i == m;
    assert i * L == m * L;
    assert i * L == |prev|;
    assert i + 1 == m + 1;
    assert (i + 1) * L == (m + 1) * L;
    assert (i + 1) * L == |repeated_str|;
    assert repeated_str[i * L .. (i + 1) * L] == repeated_str[|prev|..|repeated_str|];
    assert repeated_str[i * L .. (i + 1) * L] == str;
}
}

assert L == |str|;
assert |repeated_str| == n * |str|;
assert forall i :: 0 <= i < n ==>
    (0 <= i * |str| && i * |str| <= (i + 1) * |str| && (i + 1) * |str| <= |repeated_str|
    ↪ &&
    repeated_str[i * |str| .. (i + 1) * |str|] == str);
}
}

```

Total Token Usage

Input tokens: 6222

Output tokens: 25613

Reasoning tokens: 22728

Sum of 'total tokens': 31835

Experiment Timings

Overall Experiment started at 1780587337781, ended at 1780588035777, lasting 697996ms (698.00 seconds)

Iteration #4 started at 1780587859090, ended at 1780588035766, lasting 176676ms (176.68 seconds)

Iteration #1 started at 1780587337810, ended at 1780587565286, lasting 227476ms (227.48 seconds)

Iteration #2 started at 1780587565290, ended at 1780587699396, lasting 134106ms (134.11 seconds)

Iteration #3 started at 1780587699397, ended at 1780587859087, lasting 159690ms (159.69 seconds)